

MULTI TP6

Siyuan CHEN
Xiru ZHANG

March 2024

Table des matières

1	C. Composants périphériques	2
1.1	Question C1	2
1.2	Question C2	2
1.3	Question C3	2
1.4	Question C4	2
2	D. Lancement des tâches	3
2.1	Question D1	3
2.2	Question D2	3
2.3	Question Q3	3
2.4	Question D4	3
3	E. Activation du Timer	4
3.1	Question E1	4
3.2	Question E2	4
3.3	Question E3-E5	4
3.4	Question E6	5
3.5	Question E7	5
3.6	Question E8	5
3.7	Question E9	6
4	F. Activation des interruptions TTY	6
4.1	Question F1	6
4.2	Question F2	6
4.3	Question F3	6
4.4	Question F4	6
4.5	Question F5	7
4.6	Question F6	7
4.7	Question F7-8	7

1 C. Composants périphériques

1.1 Question C1

- Il est une cible parce qu'il répond à la requête venant du processeur pour générer périodiquement l'interruption.
- **nproc** : Le nombre de timers indépendants.
- Il existe 4 registres adressables, qui sont :
 - **TIMER_VALUE (0x0)** : Ce registre contient la valeur actuelle du timer correspondant. S'il passe à 0, une interruption est déclenchée.
 - **TIMER_RUNNING (0x4)** : Ce registre est pour contrôler le timer correspondant.
 - **TIMER_PERIOD(0x8)** : Ce registre est pour fixer une période pour le timer correspondant.
 - **TIMER_IRQ (0xc)** : Ce registre est utilisé pour vérifier l'existence de l'interruption.

1.2 Question C2

- Parce que le PibusIcu ne peut pas s'activer automatiquement, il est activé par un signal d'interruption, et donne une réponse au processeur.
- **nirq** : Définir le nombre d'entrées d'IRQ.
- **nproc** : Le nombre de timers indépendants.
- Pour un multi-processeurs, chaque processeur k correspond à une sortie IRQ_OUT[k]. Pour une entrée IRQ_IN[i], le PibusIcu contient un registre de masque spécifique pour l'entrée, ce qui permet au système d'exploitation de décider à quel processeur l'entrée va être transmise.
- Il contient 5 registres adressables, qui sont :
 - **ICU_INT (0x00)** : Enregistrer les états des 32 interruptions, soit activées soit désactivées.
 - **ICU_MASK (0x04)** : Enregistrer les interruptions masquées.
 - **ICU_MASK_SET (0x08)** : Masquer une interruption.
 - **ICU_MASK_RESET (0x0C)** : Dém masquer une interruption.
 - **ICU_IT_VECTOR (0x10)** : Il contient l'index de l'interruption active qui a l'index le plus petit.

1.3 Question C3

Dans le segment de l'ICU, l'accès aux registres VECTOR, INT, MASK, MASK_SET et MASK_CLEAR se fait en calculant par les 5 derniers bits. Si l'adresse n'est pas alignée, l'accès aux registres sera plus coûteux.

1.4 Question C4

- **Ligne 0** : Connectée à l'interruption DMA
- **Ligne 1** : Connectée à l'interruption IOC

- **Ligne n%2=0** : Connectée à l'interruption TIMER du processeur.
- **Ligne n%2=1** : Connectée à l'interruption TTY du processeur.

2 D. Lancement des tâches

2.1 Question D1

Le pointeur de pile (\$29) est initialisé à la base de la pile (`seg_stack_base`) avec une taille de pile doublée (128K) par rapport à PROC(0) (64K).

Le registre de statut (SR) est défini avec la même valeur que pour PROC(0), soit 0x0000FF13, ce qui configure les mêmes modes d'opération et de gestion des interruptions.

Le registre EPC est mis à jour avec l'adresse du point d'entrée du programme utilisateur (`main[1]`), en utilisant un décalage de 4 octets pour cibler l'instruction spécifique à PROC(1).

```
# initializes stack pointer for PROC[1]
la    $29,    seg_stack_base
li    $27,    0x20000                # intervall = 2*stack size = 128K
addu   $29,    $29,    $27          # $29 <= seg_stack_base + 128K

# initializes SR register for PROC[1]
li    $26,    0x0000FF13
mtc0   $26,    $12                  # SR <= 0x0000FF13

# jump to main in user mode: main[1]
la    $26,    seg_data_base
lw    $26,    4($26)                # $26 <= main[1]
mtc0   $26,    $14                  # write it in EPC register
eret
```

FIGURE 1 – Initialization du pointeur de pile, registre SR, EPC du processeur (1)

2.2 Question D2

L'adresses de la fonction `main_prime()` : 0x4012e8

L'adresses de la fonction `main_pgcd()` : 0x4013fc

2.3 Question Q3

Les deux "main" sont créées par deux constructeurs. On les met au début du fichier `.ld` dans le segment de données pour que, au niveau de l'édition de liens, le compilateur puisse placer ces deux fonctions au début du segment de données.

2.4 Question D4

Comme le TTY n'a pas été initialisé dans `reset.s`, il est nécessaire de vérifier et d'implémenter le processus d'initialisation correct pour le matériel lié à l'entrée.

3 E. Activation du Timer

3.1 Question E1

GIET lit le Cause registre qui détermine le raison d'entrer du noyau est interruption, appel-système ou exception.

Ensuite, il saute à la segment du gestionnaire de l'interruption.

Dans le gestionnaire d'interruption, le noyau sauvegarde le statut du processeur puis saute à `irq_handler.c`.

3.2 Question E2

`_isr_timer` execute périodiquement IRQ, et afficher sur TTY.

3.3 Question E3-E5

Les interruptions, TIMER et le composant ICU sont complétés comme ci-dessous :

```
proc0:
) # initialises interrupt vector entries for PROC[0]
L   la    $26,    _isr_timer
2   la    $27,    _interrupt_vector
3   sw    $26,    8($27)                # _interrupt_vector[2] <- _isr_timer
4
5   la    $26,    _isr_tty_get
5   sw    $26,    12($27)
7
3   #initializes the ICU[0] MASK register
)   la    $26,    seg_icu_base
)   li    $27,    0b100
L   sw    $27,    8($26)
2
3   # initializes TIMER[0] PERIOD and RUNNING registers
4   la    $27,    seg_timer_base
5   li    $26,    50000
5   sw    $26,    0x8($27)
7   li    $26,    1
3   sw    $26,    0x4($27)
)
) # initializes stack pointer for PROC[0]
L   la    $29,    seg_stack_base
2   li    $27,    0x10000                # stack size = 64K
3   addu   $29,    $29,    $27          # $29 <= seg_stack_base + 64K
4
5   # initializes SR register for PROC[0]
5   li    $26,    0x0000FF13
7   mtc0   $26,    $12                  # SR <= 0x0000FF13
3
) # jump to main in user mode: main[0]
)   la    $26,    seg_data_base
L   lw    $26,    0($26)                # $26 <= main[0]
2   mtc0   $26,    $14                  # write it in EPC register
3   eret
4   nop
5
```

FIGURE 2 – Processeur (0)

```

5 proc1:
7     # initialises interrupt vector entries for PROC[1]
3     la    $26,    _isr_timer
3     la    $27,    _interrupt_vector
3     sw    $26,    16($27)          # _interrupt_vector[4] <- _isr_timer
1
2     la    $26,    _isr_tty_get
3     sw    $26,    20($27)
4
5     #initializes the ICU[1] MASK register
5     la    $26,    seg_icu_base
7     addiu $26,    $26,    32
3     li    $27,    0b10000
3     sw    $27,    8($26)
3
1     # initializes TIMER[1] PERIOD and RUNNING registers
2     la    $27,    seg_timer_base
3     addiu $27,    $27,    0x10      # TIMER[1]
4     li    $26,    1000000
5     sw    $26,    0x8($27)
5     li    $26,    1
7     sw    $26,    0x4($27)
3
3     # initializes stack pointer for PROC[1]
3     la    $29,    seg_stack_base
1     li    $27,    0x20000          # intervall = 2*stack size = 128K
2     addu  $29,    $29,    $27      # $29 <= seg_stack_base + 128K
3
4     # initializes SR register for PROC[1]
5     li    $26,    0x0000FF13
5     mtc0  $26,    $12              # SR <= 0x0000FF13
7
3     # jump to main in user mode: main[1]
3     la    $26,    seg_data_base
3     lw    $26,    4($26)           # $26 <= main[1]
1     mtc0  $26,    $14              # write it in EPC register
2     eret

```

FIGURE 3 – Processeur (1)

3.4 Question E6

Le processeur 0 écrit la première valeur dans le vecteur d'interruption au cycle 52.

Le registre MASK[0] de l'ICU est configuré au cycle 89.

Le TIMER[0] est configuré au cycle 86.

3.5 Question E7

Le processeur 0 reçoit la première interruption du TIMER[0] au cycle 84.

L'interruption est acquittée par l'ISR au cycle 50 089.

3.6 Question E8

Lors de la première interruption par le Timer, le traitement par le processeur 0 s'effectue comme suit :

- Au cycle 1 303, le processeur entre dans le Gestionnaire d'Interruptions, Exceptions et Trappes (Appel système).

- Au cycle 50 129, il y a un branchement vers le gestionnaire d'interruptions.
- L'adresse de la routine `_isr_timer` est 80001c1c et le traitement commence au cycle 50534.
- L'adresse de la restauration des registres est au cycle 80000274 avec la reprise du traitement au cycle 54587.

Pour le deuxième interruption :

- Entrée dans le GIET au cycle 90166.
- Branchement vers le gestionnaire d'interruption au cycle 100128.
- Début du traitement dans la routine `_isr_timer` au cycle 100329.
- Restauration des registres et reprise du programme interrompu au cycle 153939.

3.7 Question E9

Parce qu'on n'a pas configuré le périphérique TTY dans le code de boot, lors de l'appel système `tty_getw_irq()`, il appelle une fonction système `tty_read_irq` pour saisir un caractère décimal jusqu'à un LF (touche Entrée), mais cette fonction échoue à cause de la non-configuration de TTY, et l'appel système est bloqué ici.

4 F. Activation des interruptions TTY

4.1 Question F1

Ce mécanisme asynchrone permet d'exécuter le programme en multitâche. Imaginez que vous voulez télécharger un gros fichier sur un navigateur, et en même temps, vous voulez taper un caractère sur l'éditeur de texte. Si l'interruption est synchrone, le téléchargement du fichier serait suspendu jusqu'à ce que vous ayez tapé toute la chaîne de caractères, ce qui causerait une perte de temps ou le risque de perdre une partie du fichier déjà téléchargé.

4.2 Question F2

Les fonctions appelées sont :

- `tty_getw_irq`
- `sys_call(SYS_CALL_TTY_READ_IRQ)`
- `_sys_handler`
- `restore`

4.3 Question F3

Un caractère est perdu si le tampon est plein lors de l'exécution de l'ISR.

4.4 Question F4

Cette fonction retire un caractère depuis le buffer noyau `tty_get_buf[tty_index]`, puis l'écrit dans le buffer utilisateur et remet à zéro l'adresse du buffer d'où le

caractère a été retiré.

4.5 Question F5

Les caractères traités sont :

- caractère décimal
- caractère LF et CR
- caractère DEL

Si la chaîne de caractère saisie est trop long, il va vider le string et mettre le string 0.

4.6 Question F6

Déclarées avec l'attribut volatile, ces deux variables informent le compilateur de ne pas optimiser leur utilisation lors de la compilation. Ces variables pourraient être modifiées en dehors du programme, par exemple par l'interruption d'un périphérique. Elles sont stockées dans le segment de données du noyau et sont déclarées comme non cachables parce qu'on souhaite accéder à la valeur depuis le registre du périphérique de manière synchrone, en temps réel et en cohérence avec le périphérique.

4.7 Question F7-8

```
proc0:
# initialises interrupt vector entries for PROC[0]
la    $26,    _isr_timer
la    $27,    _interrupt_vector
sw    $26,    8($27)           # _interrupt_vector[2] <- _isr_timer

la    $26,    _isr_tty_get
sw    $26,    12($27)

#initializes the ICU[0] MASK register
la    $26,    seg_icu_base
li    $27,    0b1100           #0b100
sw    $27,    8($26)

# initializes TIMER[0] PERIOD and RUNNING registers
la    $27,    seg_timer_base
li    $26,    50000
sw    $26,    0x8($27)
li    $26,    1
sw    $26,    0x4($27)
```

FIGURE 4 – Processeur (0)

```

proc1:
    # initialises interrupt vector entries for PROC[1]
    la    $26,    _isr_timer
    la    $27,    _interrupt_vector
    sw    $26,    16($27)          # _interrupt_vector[4] <- _isr_timer

    la    $26,    _isr_tty_get
    sw    $26,    20($27)

    #initializes the ICU[1] MASK register
    la    $26,    seg_icu_base
    addiu $26,    $26,    32
    li    $27,    0b110000          # 0b10000
    sw    $27,    8($26)

    # initializes TIMER[1] PERIOD and RUNNING registers
    la    $27,    seg_timer_base
    addiu $27,    $27,    0x10          # TIMER[1]
    li    $26,    1000000
    sw    $26,    0x8($27)
    li    $26,    1
    sw    $26,    0x4($27)

```

FIGURE 5 – Processeur (1)